

Retrieval-Augmented Generation (RAG)

A Quick Read

A concise, learner-friendly walkthrough of how RAG works, how to build one, and how to evaluate it.

Why RAG exists

LLMs are great at writing fluently, but they only know what was in their training data. They cannot see your company's PDFs, last quarter's earnings call, or a regulatory filing published yesterday. That is a problem in domains like finance, healthcare, and legal, where most of the useful information lives in unstructured documents.

RAG fixes this by giving an LLM access to an external knowledge base at query time. Instead of relying on memory, the model retrieves relevant passages from your documents and uses them as context to answer the question.

In one line: *RAG = retrieval (find the right text) + generation (write a grounded answer).*

What you get from RAG

1. **Grounded answers.** Responses are tied to real source material, not the model's parametric memory.
2. **Up-to-date knowledge.** Update the document store, and the system “knows” the new information immediately — no retraining.
3. **Fewer hallucinations.** When the model is instructed to answer only from retrieved context, it has less room to invent things.
4. **Domain adaptability.** Swap the knowledge base to move from finance to healthcare to internal HR docs.
5. **Auditability.** You can show which passage produced the answer — important for regulated industries.

The RAG workflow at a glance

A RAG system has two phases. The diagram below shows both phases on one canvas — indexing builds the searchable knowledge base ahead of time; querying happens every time a user asks a question.

Retrieval-augmented generation (RAG) is a technique that enhances generative AI models by incorporating external data sources to improve accuracy and relevance in text generation

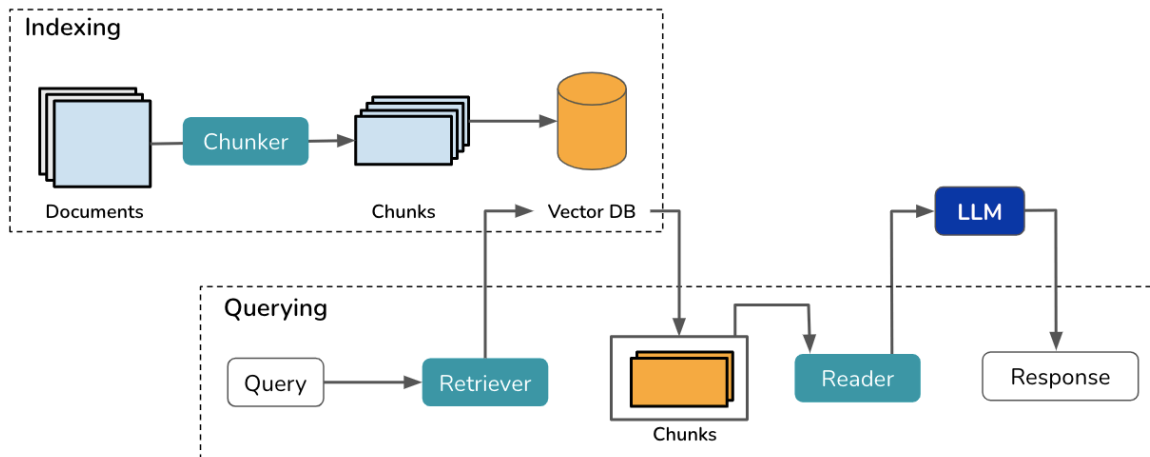


Figure 1: The full RAG workflow — indexing (top) and querying (bottom).

Indexing (done once, ahead of time)

Documents → chunked into smaller pieces → each chunk converted to a vector via an embedding model → stored in a vector database.

Querying (done per user question)

User question → converted to a vector by the same embedding model → similarity search against stored chunks → top- k most similar chunks retrieved → chunks + question stuffed into a prompt → LLM generates the final answer using only that context.

Step 1: Building the vector database (Indexing)

Two decisions matter most here: **the embedding model** and **the chunking strategy**.

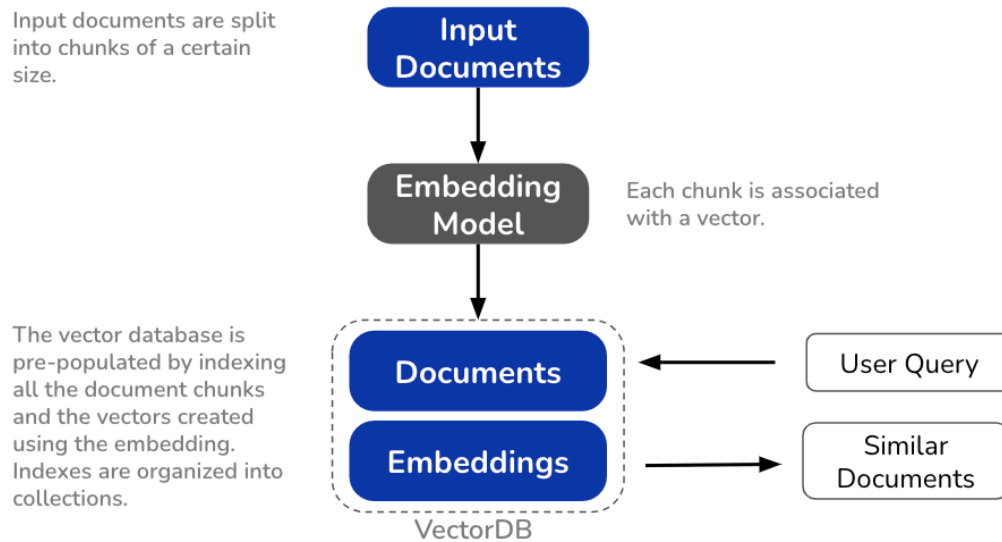


Figure 2: Building the vector database — documents are chunked, each chunk is embedded, and the (chunk, vector) pairs are stored together.

Choosing an embedding model

An embedding is a numerical vector that captures the *meaning* of a piece of text. Two sentences with similar meaning will have vectors that point in similar directions. Cosine similarity is the usual way to measure this.

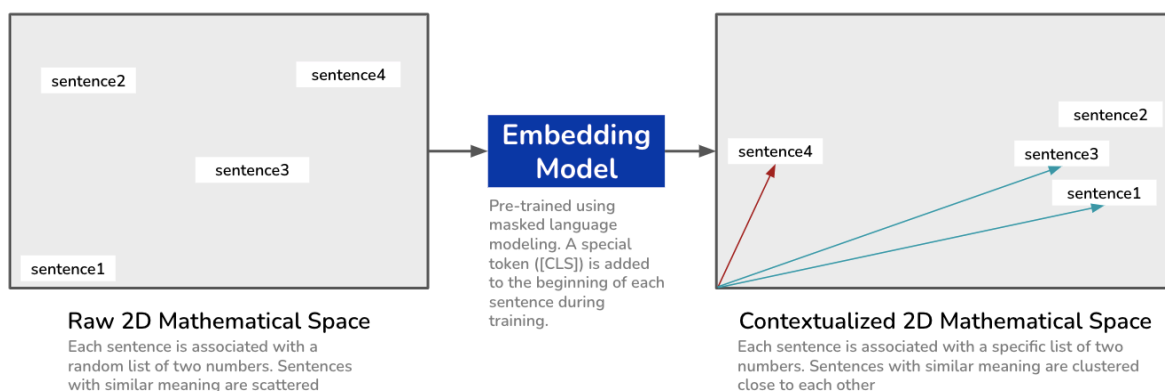


Figure 3: A good embedding model maps sentences with similar meaning to nearby points in vector space.

A few rules of thumb:

- For RAG specifically, use a model trained on sentence-pair similarity — e.g., **Sentence-BERT** variants like **all-MiniLM-L6-v2**, **BGE** models like **BAAI/bge-base-en**, or proprietary APIs like OpenAI's **text-embedding-3-small/large** or Cohere Embed.
- Open-source models are not inherently worse than paid ones — check the **MTEB leaderboard** on Hugging Face and pick a model that performs well on retrieval tasks for your language and domain.
- For multimodal retrieval (text + images), use **CLIP** or similar vision-language models.

Chunking documents

Embedding models have a fixed maximum input length. For many BERT-based models, the cap is 512 tokens; newer models (e.g., **gte-large-en-v1.5**, OpenAI **text-embedding-3-large**) support 8K–32K tokens. Text longer than the cap gets truncated, so you must split documents into chunks.

The standard approach: **fixed-size chunks with a small overlap**. For example, 512-token chunks with a 16-token overlap.

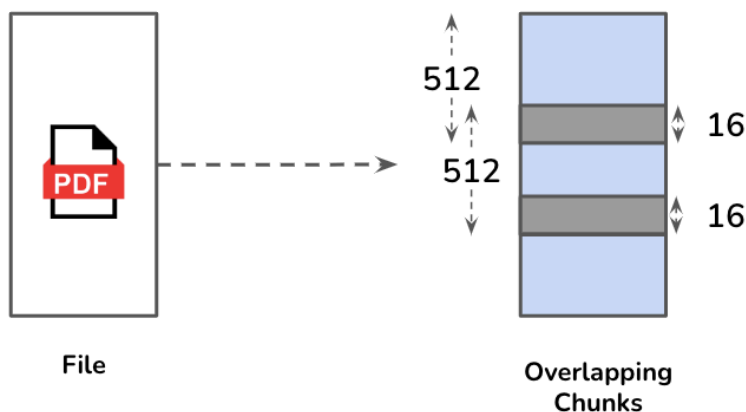


Figure 4: Fixed-size chunks (512 tokens) with a small overlap (16 tokens) between adjacent chunks.

Why overlap? Because important information sometimes straddles a chunk boundary. Overlap preserves continuity so a relevant sentence is not cut in half and lost.

A smarter alternative when document structure allows: **semantic chunking** by natural sections (e.g., “Risk Factors,” “Legal Proceedings” in a 10-K filing). This usually retrieves better than blind fixed-size splits.

Each chunk + its vector goes into the vector store — e.g., **ChromaDB**, **Pinecone**, **Weaviate**, or **FAISS**. These are full databases — they support create, read, update, and delete operations.

Step 2: Querying

When a user asks a question, the system does the following:

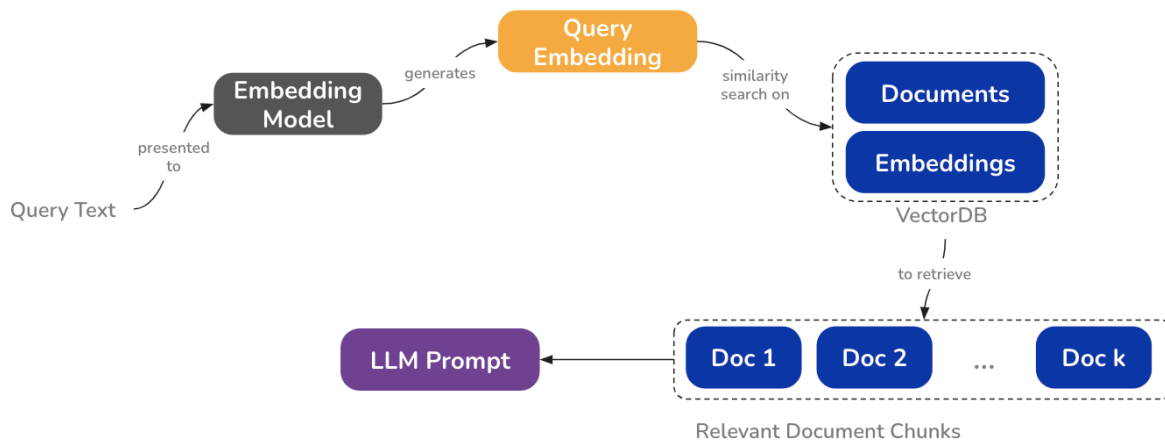


Figure 5: Query-time flow — the question is embedded, used for similarity search, and the top- k chunks are passed to the LLM as context.

1. Embed the question using the **same model** used for indexing. (Critical — different models produce incompatible vector spaces.)
2. Run a similarity search against the vector store.
3. Pull the top- k chunks (often $k = 3$ to 10).
4. Inject them into a prompt template that looks roughly like the example below.
5. Send to the LLM and return the answer.

Prompt template (illustrative):

Answer the following question using only the context below. If the answer is not in the context, say so. Context: {retrieved chunks} Question: {user query}

Frameworks like **LangChain** and **LlamaIndex** provide the abstractions (chunkers, retrievers, readers) that wire all this together.

Evaluating a RAG system

A RAG system has two phases (indexing and querying), and the querying phase itself has two sub-components: **retrieval** and **generation**. Evaluation needs to cover both. Three standard metrics:

1. **Faithfulness / Groundedness** — Is the answer actually supported by the retrieved context, or did the model make things up?
2. **Contextual Relevancy** — Did retrieval pull chunks that are actually relevant to the question?
3. **Answer Relevancy** — Does the final answer actually address the user's question?

The usual methods: build a small set of gold question–answer pairs with human annotators, then either score new outputs against them or use **LLM-as-a-judge** (a strong LLM grading the RAG system's outputs using a rubric). Frameworks like **RAGAS** and **DeepEval** implement these metrics out of the box.

In summary

RAG pairs the fluency of a generative LLM with the precision of a retrieval system. By chunking your documents, embedding them, and pulling the most relevant pieces at query time, you get answers that are grounded, up to date, and traceable to a source — without retraining the model. Get the embedding model and chunking right, evaluate both retrieval and generation, and you have a production-ready foundation for document Q&A.